AIC · 智能体互联技术报告

AegisMesh 智能体互联协作平台

基于 AIP-PUB 官方开源代码与 AIP 国家标准构建的质量感知多智能体互联协作运行时：智能体注册同步、能力发现排名、六维质量路由、三种协同模式、国标群组双传输与共享画布协作，覆盖从注册到访问的完整链路。

技术部分 · 遵循 AIP 国家标准 · 2026 年 9 月

AegisMesh · 智能体互联协作平台 — 技术报告（技术部分）

依据《AIC · 智能体互联技术报告参考大纲》（附件2）编写。本文档只覆盖技术内容，所有数据均来自项目仓库与线上运行库的实测值；尚不具备的数据以「待补」显式标注，不做推测性描述。

一、作品概述

1. 行业与技术背景

智能体互联行业正处于从"单体智能"走向"群体协同"的转折期。国家层面的智能体互联标准 AIP（ACPs 协议族，含 ACS 能力描述、AIP 任务协议、ADP 发现协议）为异构智能体之间的互操作提供了统一口径：智能体以 AIC 身份码标识、以 ACS 元数据声明能力、以 JSON-RPC/SSE/消息队列等传输方式完成任务往返。

行业现状的三个客观事实：

1. **协议落地工具链分散**。AIP-PUB 开源社区（https://github.com/AIP-PUB、https://atomgit.com/AIP-PUB）提供了 acps-sdk、acps-cli、registry-server、ca-server、discovery-server、mq-auth-server 等组件，但组件之间需要使用者自行组装：注册要走注册平台 REST API、证书要走 EAB→ACME 流程、发现要对接发现服务器，缺少一个把这些环节串成"开箱可用的运行时"的整合层。
2. **智能体执行质量参差**。标准解决了"怎么连上"，没有解决"连上之后任务交给谁"——异构智能体在能力匹配度、历史可靠性、响应延迟、负载水平上差异很大，调用方需要质量感知的调度。
3. **协同模式单一**。多数开源示例只覆盖单轮点对点调用，缺少并行分工、串行接力、集群研讨等可复用的协同编排范式，也缺少多智能体共同维护一份产物的协作界面。

本作品（AegisMesh）的定位即针对上述缺口：**在官方开源代码之上，构建一个质量感知的多智能体互联协作运行时，把"注册—发现—互联—访问—协同—观测"整条链路产品化。**

2. 作品目标定位

核心研发目标：基于 AIP-PUB 开源代码（ACPs SDK v2.2.0、acps-cli），遵循 AIP 标准，实现一个支持智能体注册同步、能力发现（带排名）、质量路由、多模式协同调度、共享画布协作与国标群组通信的多智能体互联平台。

技术指标（实测达成情况）：

指标	目标	实测
AIC 身份码兼容	符合 10 段编码、可通过结构校验	通过： 平台内全部智能体 AIC 通过 `validate_aic_format` 校验
ACS 能力描述	发布可被外部读取的 acs.json	通过： 平台级 `/.well-known/acs.json` + 每智能体 `/api/agents/{id}/acs`

指标	目标	实测
注册与审核同步	对接注册平台，状态可回读	通过： DRAFT/PENDING/APPROVED/REJECTED 全状态同步
mTLS 双向认证	国标 CA 签发证书、身份绑定	通过： 平台 CA 签发 serverAuth/clientAuth 证书；入站出站均已实现
发现排名	候选智能体带质量评分	通过： 发现阶段六维评分排序
多模式协同	并行/串行/集群三种范式	通过： 三种模式均可用
线上运行	稳定部署与访问	通过： systemd 常驻、155 项后端测试、56 次真实任务调用

硬性目标（梧桐平台全流程验证）完成状态：

验证场景	状态	佐证
注册	通过： 已完成	智能体 AIC `1.2.156.3088.1.BUPT.L2K8YU.9FTJ5K.FUG31M.0VW4` 已在注册平台注册并通过审核（平台 CA 已为其签发证书，证书签发以审核通过为前提）；注册/审核状态可通过平台 API 回读
发现	通过： 已完成	"智能体互联协作枢纽"预检中 AIP-ADP-DISC-000 可发现性 6/6 命中（PASS）
互联访问	通过： 通道已建立	mTLS 监听（`aip.demmo.cc.cd:8443`）已出示平台 CA 签发的 serverAuth 证书；平台级 RPC 端点已在 ACS `endPoints` 声明
访问量后台数据	**待补**	需从梧桐平台后台导出访问量截图与日志（平台侧数据，本作品无法自行生成）

开源情况：项目已完成双端 git 版本管理；计划按大赛要求推送至 AIP-PUB 开源社区（**待执行**）。

二、需求分析

1. 行业痛点剖析

结合梧桐平台接入的实际开发过程，本作品面对的核心痛点包括：

1. **协议不统一**：不同框架的智能体对外暴露的接口形态各异（HTTP API、SSE、消息队列、自有 RPC），没有统一身份与能力描述时无法互相发现与调用。
2. **跨平台互通的安全成本高**：AIP 采用 mTLS + AIC 身份绑定，证书链路（注册审核 → AIC → EAB 凭据 → ACME 签发 → 信任包 → 双向认证）环节多、工具链分散，手工完成易错。
3. **标准化落地难度大**：标准的载荷是通用结构（TaskCommand/TaskResult），高级协同语义（并行分工、串行接力、集群研讨、共享文档编辑）需要使用者在载荷约定层面自行设计。
4. **调用质量不可控**：标准化的发现只回答"有哪些智能体"，不回答"该把任务交给谁"——缺少基于能力匹配、可靠性、延迟、负载的量化调度。

由此明确本作品要解决的四个关键问题：**注册与证书链路的产品化、发现结果的可用性排序、多范式协同调度、运行过程的可观测。**

2. 场景与功能需求调研

目标场景选定科研协作（文献综述、实证设计、论文评审等多专家接力/研讨），通过平台适配测试明确了以下需求，并划分四类指标：

功能指标：

- 智能体注册同步（含 ACS 上传、审核状态回读）；
- 三种发现方式中的两种（本地 ACS List 带排名、发现服务器 ADP）；
- 三种协同模式（并行分工 / 串行工作流 / 集群研讨）与点名词度；
- 共享画布（多智能体分段编辑同一报告，版本冲突控制）；
- 国标群组通信桥（本地黑板与 MQ 群组双传输）；
- 全链路可观测（轨迹事件、路由评分、工具调用、画布修订）。

性能指标（实测口径见第五部分）：

- 单任务端到端延迟、流式首字时间、并发派发数量（asyncio 并发模型）、探针巡检频率（30s/轮）。

兼容性指标：

- 模型层兼容 OpenAI 兼容接口 / OpenAI Responses / Anthropic Messages 三种传输；
- 协议层兼容 AIP RPC、SSE 流式、MQ 群组三种交互方式；
- 外部智能体无需改造即可经 mTLS + AIP RPC 参与协同。

可部署指标：

- 单机 systemd 常驻（主服务 + mTLS 监听 + 日志采集三个服务单元）；
- nginx 终结公网 TLS、内网反代；一键构建与同步部署脚本；
- SQLite 单文件存储，零外部数据库依赖（消息队列仅国标群组通道需要）。

三、智能体互联技术选型与方案设计

1. 基础开发资源选型

- **官方开源代码**：ACPs SDK v2.2.0 与 acps-cli（锁定 commit `3985c13`），包含协议模型（ACS/AIP/ADP/群组/通知/流式）、`AcmeClient`、`GroupLeaderMqClient` 等官方实现；
- **梧桐平台接入环境**：registry-server（注册与审核，`/auth/login`、`/agent/client`、`/atr/eab/{aic}`）、ca-server（ACME，EAB 绑定签发）、discovery-server（ADP 发现协议）；
- **开发环境**：Python 3.13 + FastAPI + SQLAlchemy（后端），React 19 + TypeScript + Vite（前端），部署于 GCP VM（asia-east2，Ubuntu），nginx 终结 TLS；

• **遵循思路**：协议原语一律复用官方 SDK（身份校验、ACS 模型、AIP 客户端、ACME、群组客户端、ADP），平台侧只做应用层组合与产品化，不自造协议字段。

2. 核心技术方案设计

整体架构（四层）：

```
├── 接入层：nginx（demmo.cc.cd 公网 TLS） + AIP mTLS 监听（aip.demmo.cc.cd:8443，平台 CA 证书）
├── 应用层：FastAPI
│   ├── 编排器：质量路由 / 点名直派 / 并行分工 / 串行接力 / 集群研讨
│   ├── 传输层：OpenAI 兼容 · OpenAI Responses · Anthropic · AIP RPC（工具循环统一）
│   ├── 协议适配：注册客户端 · EAB/ACME 证书 · ADP 发现 · 群组 MQ 桥 · mTLS 中间件
│   ├── 共享状态：画布（版本化文档） · 协作黑板 · 轨迹事件
│   └── 观测：探针巡检 · AMP 日志（access/heartbeat 等，fluentbit 转发）
├── 数据层：SQLite（业务库 + 画布/轨迹/注册状态）
└── 前端：React（会话列表 · 协作面板 · 画布标签 · 管理后台）
```

关键技术选型：

• **注册发现机制**：注册走官方 REST API 产品化封装；发现采用"本地 ACS List（内部注册表）+ ADP 发现服务器"双源，**在发现阶段完成六维质量评分与排序**，发现结果即带排名的候选名单；

• **跨智能体数据传输**：托管智能体走平台内工具调用；外部国标 Partner 走标准 TaskCommand/TaskResult 载荷（文本 + `StructuredDataItem` 结构化 JSON），不做协议扩展；

• **协同调度算法**：六维加权质量评分（能力匹配、历史可靠性、响应延迟、实时负载、健康状态、成本），支持点名词度直派（不重复评分）与自动调度（按分排序）；

• **安全交互鉴权**：EAB → ACME 证书签发（clientAuth/serverAuth 双用途）、mTLS 双向认证 + AIC 身份绑定、API Key/管理员会话/聊天口令分层访问控制、供应商密钥加密存储。

原生复用模块 vs 自主开发模块：

类别	模块
原生复用（官方开源）	acps-sdk 协议模型与校验（AIC/ACS/ADP/Task）、`AipRpcClient`、`GroupLeaderMqClient`/`GroupPartnerMqClient`、`AcmeClient` 与密钥工具、`AipPeerCertificateMiddleware`、ADP 客户端
自主开发	六维质量路由器与策略管理、协调者工具链（`list_agents` 排名发现、`dispatch_task`、`execute_workflow`、`run_swarm_discussion`、`check_subagent_status`、`fetch_session_history`）、共享画布（版本链 + 提示词协议）、注册/证书产品化 API、多模式会话前端（协作面板、群聊、画布）、完整日志查看器、流式会话绑定

四、项目实施

1. 整体实施方案

实施流程按"需求拆解 → 开源代码适配改造 → AIP 标准兼容开发 → 梧桐平台对接调试 → 全功能联调测试"推进：

1. 集成 acps-sdk 与 acps-cli（锁定官方仓库同一提交），完成 AIC 校验与 ACS 模型对齐；
2. 构建智能体运行时（三种模型传输 + 工具循环）与质量路由；
3. 实现协调者工具链与三种协同模式，配套协作面板前端；
4. 对接梧桐 registry-server（注册/审核/状态同步）与 ca-server（EAB/ACME 证书签发）；
5. 打通 mTLS 入站监听与出站身份绑定，完成发现与互联访问验证；
6. 实现国标群组双传输桥与画布协作模式，全量回归。

开发周期：2026-09-12 起持续迭代（首批证书 09-22 签发；梧桐预检 09-25 完成并整改端点声明与 serverAuth 证书两项 HIGH 缺口）。软硬件资源：GCP 单实例（Ubuntu 22.04，2 vCPU/4GB 起步即可承载），本地开发 macOS + Node 20 + Python 3.13。团队成员分工：**（待团队成员自行填写）**。

2. 关键技术落地实现

（1）注册与审核同步（backend/app/services/registration.py）

对接 registry-server 用户侧 API：`/auth/login`（OAuth2 表单）→ `POST /agent/client`（建草稿，附完整 ACS）→ `/agent/client/{id}/submit`（提交审核）→ `GET /agent/client/{id}`（状态回读）。官方审核状态 DRAFT/PENDING/APPROVED/REJECTED 映射为平台内注册状态，审核意见随状态存档。

（2）EAB → ACME 证书签发（backend/app/services/certificates.py）

以官方 acps-cli 的 `AcmeClient` 为基础：向注册服务 `POST /atr/eab/{aic}` 换取一次性 EAB 凭据 → 生成 ACME 账户密钥（EC）与智能体密钥（Ed25519）→ CSR → `new_account`（外部账户绑定）→ `new_order` → `finalize` → 轮询取证书。按用途区分文件（clientAuth / serverAuth），服务端监听使用 serverAuth 证书，出站调用使用 clientAuth 证书。

（3）mTLS 监听与身份绑定（scripts/serve.py + AipPeerCertificateMiddleware）

平台 mTLS 监听（8443）出示梧桐 CA 签发的 serverAuth 证书；握手策略为 CERT_OPTIONAL（互操作预检可在握手层读取服务端证书），实际 AIP 命令的身份校验在应用层执行：请求方证书 AIC 必须与 TaskCommand 的 senderId 一致，覆盖 `/aip/{id}/rpc`、`/aip/{id}/stream`、`/aip/{id}/notification/*` 全部托管端点。

（4）发现阶段质量评分（backend/app/services/orchestrator.py `_list_agents`）

发现工具返回的每个候选（本地注册表 + ADP 全网发现）均附带六维评分、资格说明，按分排序。评分公式：score = Σ(w□ · s□)，其中能力匹配基于查询与技能的中英文分词覆盖度，历史可靠性采用拉普拉斯平滑 (成功+2)/(总数+4)，延迟与负载采用倒数归一，权重可在管理端调整且强制合计 100%。主智能体参考排名自行选择派发对象；**用户指名道姓时由协调者直接点名派发，不再重复评分。**

（5）三种协同模式（同一套派发通道，平台串行执行）

- 并行分工：`dispatch_task(wait=false)` 并发派发；

- 串行工作流：execute_workflow(steps)，前棒交付物自动注入后棒输入，形成棒次链；
- 集群研讨：run_swarm_discussion(topic, participants, rounds)，发言逐条写入共享黑板。

（6）画布协作（提示词协议，零协议扩展）（backend/app/services/canvas.py）

共享画布为平台持有的版本化文档实体（CanvasDoc/CanvasRevision）。托管智能体通过注入的 canvas_read / canvas_write_section / canvas_append / canvas_history 工具编辑；外部国标 Partner 无工具注入机制，采用**提示词协议**：画布快照随 TaskCommand 下发，Partner 在回复文本中输出标准 JSON 编辑块（canvas_write_section），平台解析、版本校验后应用。冲突控制采用 base_version 校验 + 平台串行应用。

（7）兼容性问题与优化（真实案例）

- TestClient 场景下内存 SQLite 跨线程不可见 → 测试基座改用临时文件库；
- 前端 的 UA 默认 padding:1px 6px 导致 18px 图标圆钮内白 X 图标偏移 3.5px → 显式 padding:0 并改角点 transform 居中；
- tsc -b 增量缓存掩盖真实 JSX 错误 → CI 使用 --force；
- 大模型回传文本中的伪工具调用残留 → clean_model_output 扩展剥离 与全部平台工具名的 JSON 块；
- 外部 Partner 消息载荷假设错误（包装结构 vs 裸 TaskResult）→ 以兼容写法 getattr(message, "result", None) or message 修正，并补 mock 测试；
- 流式输出串台 → 运行任务绑定发起会话，分离视图改为回写原会话存储。

五、应用成效

1. 功能成果展示

系统全部功能已在线上环境（demmo.cc.cd / aip.demmo.cc.cd）实测：智能体注册同步、探针巡检、质量路由与自动迁移、三种协同模式、群聊与画布协作、SSE 流式输出、完整日志查看（分页 + 全量轨迹 + 原始 JSON）。梧桐平台后台访问量截图与交互记录：**待补**（需从梧桐平台后台导出）。

2. 量化成效分析（线上运行库实测，截至报告日）

指标	数值
任务调用总数	56 次（完成 45 / 失败 11，成功率 80.4%）
完成任务平均端到端延迟	11,141.7 ms（含多棒派发与集群研讨的长任务）
路由派发记录	40 次
轨迹事件	315 条
探针巡检记录	15,344 条（30 秒/轮 × 全部启用智能体）
模型调用 / Token 消耗	111 次 LLM 调用；输入 625,181 tokens、输出 80,186 tokens
后端测试	155 项全部通过

指标	数值
代码规模	后端 Python ≈12,900 行；前端 TypeScript ≈17,000 行

三维度社会效益：技术落地（单机可部署、零外部数据库依赖的轻量运行时）；标准化推广（全部协议交互走官方 SDK，未扩展协议字段）；场景赋能（科研协作场景的多专家接力与研讨已在真实模型下跑通）。

3. 必要验证场景完成情况

必要场景	状态	说明
注册	通过：	AIC 已在梧桐注册平台注册并通过审核（平台 CA 已签发证书为佐证）；注册与审核状态可经 API 回读
自动发现	通过：	ADP 发现已对接；枢纽预检 AIP-ADP-DISC-000 六组问题语言 6/6 命中（PASS）
跨端互联访问	通过：（通道级）	平台 CA 签发的 serverAuth 证书已部署于 mTLS 监听；平台 AIP RPC 端点已在 ACS `endPoints` 声明；外部握手与 RPC 连通已验证
访问量数据留存	待补	梧桐后台访问量截图与日志待平台后台导出

说明：第三方"智能体互联协作枢纽"对该 AIC 的一致性预检中，AIC 结构（10 段、前缀 1.2.156.3088）、可发现性（6/6）通过；初始报告的"端点未声明"（HIGH）与"serverAuth 证书未测"（HIGH）两项缺口已分别通过 ACS `endPoints` 声明与平台 CA serverAuth 证书签发整改，待复跑确认。该预检为远程测试法口径，非认证结论。

4. 开放开源贡献

项目已完成 git 版本管理与改造说明文档；按大赛要求向 AIP-PUB 开源社区推送代码与文档的计划待执行。

六、总结与展望

1. 经验总结

- **协议原语优先复用官方 SDK**：身份、ACS、任务、证书、群组全部走官方实现，平台侧仅做组合，保证了与国标口径一致；
- **把语义前移到发现阶段**：候选在发现时即带质量评分与资格说明，调用阶段点名词度零成本直派，链路语义清晰；
- **外部参与的合理边界**：托管智能体可注入工具，外部 Partner 通过提示词协议在标准载荷内参与——不扩展协议也能实现高级协同；
- **诚实分级的数据面**：不具备的数据显式标注"待补"，避免演示与事实脱节。

不足：消息队列基础设施（RabbitMQ/mq-auth-server）尚未部署，国标群组通道的实弹验证依赖其就绪；任务内流式为结果切块而非 token 级；方式三（发现智能体）未实现。

2. 未来规划

1. 部署 RabbitMQ + mq-auth-server，实弹打通国标群组通道并完成外部 Partner 同群协作演示；
2. 实现方式三"发现智能体"（以自然语言包装 ADP 检索，输出智能体与技能组合建议）；
3. 模型传输层接入 token 级 SSE，配合画布实现逐字协同编辑；
4. 按 AMP 规范核对访问日志 correlationId=taskId 口径，完善监控面板；
5. 向 AIP-PUB 开源社区推送代码与文档，参与社区共建。

七、附录

1. **测试数据**：后端自动化测试 155 项（`backend/tests/`，28 个文件），覆盖编排调度、传输协议、注册证书、群组桥、画布、平台特性与鉴权；线上运行库统计口径见第五部分。
2. **完整工程源码**：`backend/`（FastAPI 应用，13 个服务模块 + 28 个测试文件）、`frontend/`（React 控制台与聊天界面）、`scripts/`（部署、证书签发、等待审核通过、Fluent Bit 配置等脚本）；改造说明见仓库 README。
3. **参考文献**：AIP 智能体互联国家标准相关文档；ACPs-community 开源仓库（SDK/CLI/registry-server/ca-server/discovery-server/mq-auth-server）；acps-sdk v2.2.0 README；梧桐智能体互联平台使用指南。
4. **佐证材料**：梧桐枢纽预检报告、平台 CA 签发证书（serverAuth/clientAuth）、线上运行截图与日志（见线上环境）；演示视频与平台访问量后台截图待补。
5. **开源贡献材料**：待推送 AIP-PUB 社区后补充仓库主页与 Commit 记录截图。

本报告基于仓库提交 `090b69d`（本地）/ `cdd2cbc`（服务器）时刻的代码与线上运行库实测数据编写；"待补"项均为外部依赖数据，非功能缺失。